ClaimFast: Prompt Strategy Evaluation for Motor Claim Triage

Case Study 1, MCA Semester I, Agentic AI. Model: openai/gpt-oss-20b on Groq. Repository: Aloy17/ClaimFast.

1. Problem statement

A motor insurer receives free-text First Notice of Loss (FNOL) messages in English and Hinglish. Each message must be classified into one of six claim types (own_damage, third_party_liability, theft, total_loss, glass_only, not_a_claim), given an urgency (high, medium, low), and mapped to seven extraction fields (policy number, incident date, location, vehicle registration, estimated amount in INR, injury reported, police report filed). Missing values must be null, and false is allowed only when the message denies the condition. The question is which prompting strategy gives the best balance of accuracy, reliability, cost and speed on the same model.

2. Data and methodology

Dataset. 80 synthetic, anonymised messages: 20 development records (*data/dev.jsonl*) used to write prompts and pick few-shot examples, and 60 held-out test records (*data/test.jsonl*) used only for evaluation. Test claim types: third_party_liability 13, own_damage 11, total_loss 10, theft 9, glass_only 9, not_a_claim 8. Urgency: medium 31, low 17, high 12. Edge cases cover Hinglish, multiple incidents, forwarded messages, missing fields and ambiguous theft. Rules are written in *data/labelling_guide.md*. Labels were written by one annotator; the independent 20-record review is not yet complete (see Limitations).

Strategies. All use the same system prompt definitions, schema and validator.

ID	Strategy	Examples	What differs
A	Zero-shot	0	Instructions and definitions only
B1	Few-shot	3	DEV001, DEV008, DEV003 added to the prompt
B2	Few-shot	5	Adds DEV010, DEV005
B3	Few-shot	8	Adds DEV006, DEV014, DEV017
C	Reasoning-guided	0	Explicit decision checks and a short evidence summary before the JSON
D	Not implemented	-	Optional in the assignment; not built

Experiment settings. Temperature 0, reasoning effort low, maximum 2,000 completion tokens, one model for every run. 5 strategies x 3 runs x 60 test records = 900 requests, run on 2026-11-06. Raw responses, token counts and latency are saved in *outputs/final_benchmark/*. All 900 returned valid JSON that passed validation (0 API errors, 0 parse errors). Few-shot examples come only from the development set.

Metrics. Per-field exact match; accuracy, macro precision, recall and F1 for claim_type and urgency; micro F1 over the seven extraction fields; record exact match (all nine fields correct); recall for injury_reported = true and for urgency = high; pairwise agreement across the three runs; latency, tokens and cost. Cost uses Groq list prices of $0.075 per million input tokens and $0.30 per million output tokens (Groq model page, read 2026-11-13). Metrics are computed offline by *analysis.py* from the saved log.

3. Results

Strategy	claim_type acc	claim_type macro F1	urgency acc	urgency macro F1	Extraction micro F1	Record exact (strict)	Record exact (location normalised)
A zero-shot	89.4%	0.911	77.2%	0.785	0.913	30.0%	57.8%
B1 k=3	95.0%	0.953	89.4%	0.892	0.890	26.7%	69.4%
B2 k=5	92.8%	0.934	98.3%	0.985	0.886	30.6%	71.1%
B3 k=8	98.9%	0.987	96.7%	0.965	0.914	44.4%	81.7%
C reasoning	96.7%	0.972	90.0%	0.898	0.901	32.2%	72.8%

Each figure pools the three runs (180 predictions per strategy). Strict record exact match is low for every strategy because the gold *location* label often keeps words such as 'near' or 'outside the gym in' while the model returns the place name (30 of 60 test records). The normalised column treats those as equal; it changes no other field.

Per-field exact match

Strategy	claim_type	urgency	policy	date	location	reg no	amount	injury	police
A zero-shot	89%	77%	94%	100%	59%	98%	98%	95%	99%
B1 k=3	95%	89%	87%	100%	50%	95%	100%	98%	98%
B2 k=5	93%	98%	85%	100%	51%	95%	98%	95%	98%
B3 k=8	99%	97%	98%	100%	60%	97%	98%	94%	97%
C reasoning	97%	90%	93%	100%	54%	97%	98%	94%	99%

Location exact match (50 to 60%) is depressed by the label-format issue above. Policy number is lowest for B1 and B2 (85 to 87%) because the models return null on not_a_claim messages that do state a policy number; B3 recovers it (98%). Incident date is 100% for every strategy.

Reliability, speed and cost

Strategy	Valid JSON	Injury recall	High-urgency recall	Mean latency (s)	p95 latency (s)	Mean prompt tok.	Mean output tok.	Cost per 1,000 claims
A zero-shot	100%	100% (27/27)	100%	3.91	8.0	848	134	$0.104
B1 k=3	100%	100% (27/27)	100%	4.74	10.8	1308	114	$0.132
B2 k=5	100%	100% (27/27)	100%	2.89	13.4	1613	120	$0.157
B3 k=8	100%	100% (27/27)	100%	7.04	16.0	2057	119	$0.190
C reasoning	100%	100% (27/27)	100%	9.19	18.1	1321	282	$0.184

Latency is wall-clock time on a shared free-tier endpoint and varies between calls; B2 is faster than A on average even though its prompt is nearly twice as long, which shows the timings carry network and queue noise. Treat latency differences under about 2 seconds as unreliable. Token counts and cost are exact.

4. Interpretation

Zero-shot against few-shot. Few-shot examples help claim_type: 89.4% (A), 95.0% (B1), 92.8% (B2), 98.9% (B3). Urgency improves more: 77.2% for A against 98.3% for B2 and 96.7% for B3. The zero-shot prompt marks 41 of 93 medium cases as high; the examples remove nearly all of that.

Do the gains flatten? Not cleanly. claim_type rises from k=0 to k=3, dips at k=5, and rises again at k=8, so the curve is not monotonic and has not flattened by k=8. With 60 records, one record is 1.7 percentage points per run, and the dip at k=5 is one to two records. The run-to-run spread is small (standard deviation under 1.5 points), but the test set is small, so differences of 2 to 3 points between neighbouring strategies should not be treated as proven. The clear signal is the large first step (A to B1 and B2) on urgency and the best result at k=8.

Reasoning-guided (C). C reaches 96.7% claim_type and 90.0% urgency without any examples, better than A on both but behind B3. It uses about 2.1 times the output tokens of A (282 against 134), has the highest mean latency (9.2 s) and costs about as much as B3 per 1,000 claims. It did not fix the policy-number or injury-denial errors.

Consistency. Pairwise agreement between runs for the same strategy and record is 94.7% overall; every field is at least 98% (incident date, registration and amount 100%). Record exact match varies by about 3 points or less between runs for any strategy, and B1 is identical across runs. At temperature 0 the outputs are close to deterministic; the remaining variation comes from the provider, not from the prompt.

Safety-critical recall. Injury recall (injury_reported = true) is 100% for every strategy (27 of 27 positive predictions across three runs), and high-urgency recall is 100% for every strategy. No injured claim was missed or downgraded in any of the 900 runs. The cost of the weaker strategies is over-escalation (A raised 41 medium cases to high), not under-triage.

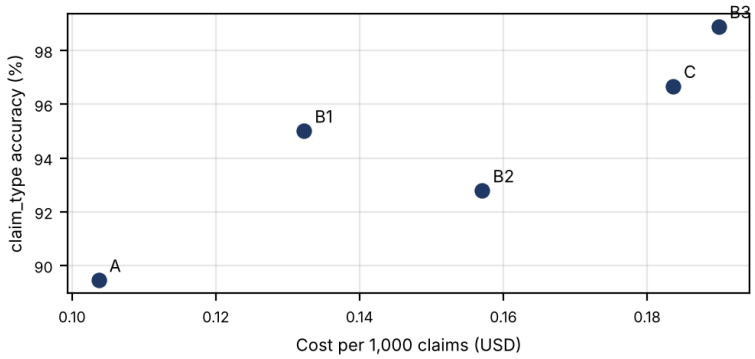
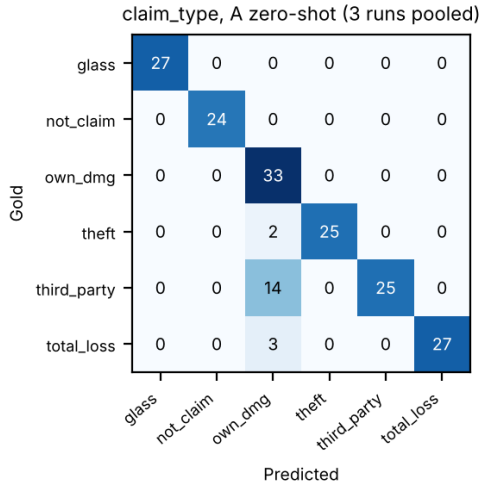
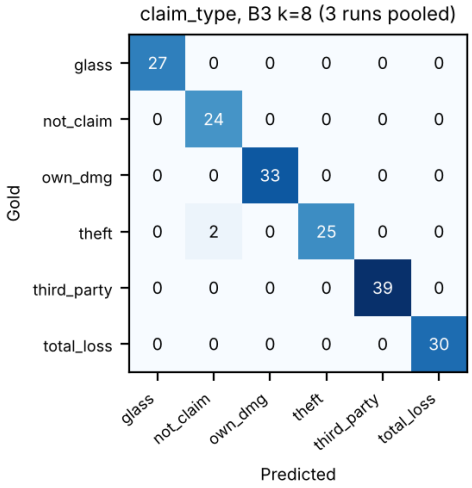


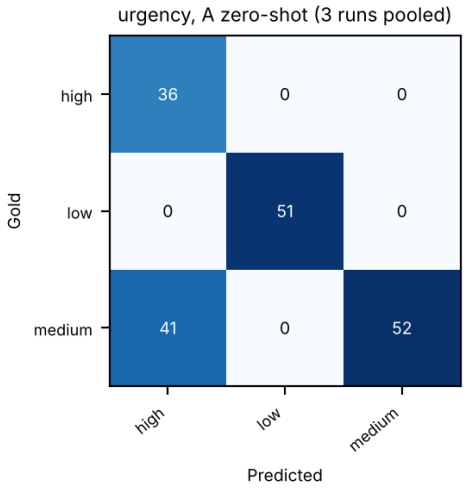
Figure 1. claim_type accuracy against cost per 1,000 claims. B3 is the most accurate and about 1.8 times the cost of A.

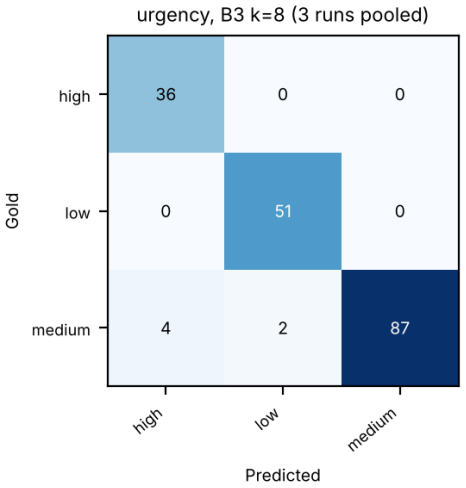
5. Confusion matrices

Counts pool the three runs. Zero-shot (A) and the best strategy (B3) are shown; matrices for every strategy are in *outputs/final_benchmark/analysis/analysis.json*.









Reading the matrices. Under A, 14 of 39 third_party_liability predictions and 3 of 30 total_loss predictions become own_damage, so own_damage precision is 63% and third_party_liability recall is 64%. B3 removes all of those. The only claim_type errors left in B3 are two predictions of TEST029 (a missing vehicle) as not_a_claim, a case the labelling guide itself flags as ambiguous. On urgency, A puts 41 medium cases into high, while B3 leaves 4 medium as high and 2 as low. No high case was missed in either strategy.

6. Error analysis

The full sheet is *outputs/final_benchmark/analysis/error_analysis.xlsx*. It lists every distinct field failure per strategy with its cause, severity, mitigation, a check of the extracted value against the source message, and whether the gold label is flagged. A: 31 rows, B1: 24, B2: 24, B3: 14 (all that exist), C: 21. No API or format failure occurred, and every extracted policy number, registration, date and amount appears in its source message, so there are no invented values.

Category	A	B1	B2	B3	C	Cause and mitigation
Urgency set to high for a destroyed vehicle	14	6	0	2	6	Model treats 'destroyed' as unsafe. Guide: high only for injury or a stated unsafe or undrivable vehicle. Keep the rule, add a total_loss example with medium.
Policy or registration null on not_a_claim	5	12	13	3	7	Prompt line says every extraction field is null for not_a_claim. Reword to 'extract values the message states'.
Third-party signal missed	5	2	4	0	2	Injured passenger or damaged other car not treated as third party. Add examples.
Explicit denial or confirmation missed	3	1	1	3	5	'Nobody injured', 'no FIR yet' returned as null. Add denial examples, including Hinglish.

Category	A	B1	B2	B3	C	Cause and mitigation
Unsupported false	0	1	3	3	0	'Only glass is damaged' read as no injury. Reinforce 'false only if explicitly denied'.
Wrong incident used	1	0	1	1	1	TEST058: Rs 12,000 windshield estimate from the withdrawn first incident. Add a multi-incident example.
Suspected theft as not_a_claim	0	2	2	2	0	TEST029. Guide marks it ambiguous. Add a missing-vehicle example.
Claim type confused (theft or total loss)	2	0	0	0	0	A only: parts stolen read as own_damage; 'beyond repair' missed.

Model mistakes against label and metric issues. The location mismatch (30 records) is a label convention, not a model error; it is listed separately and excluded from the failure count. TEST029 and TEST051 are borderline gold labels. Urgency over-escalation on total loss is a model-versus-guide disagreement that a reader could defend either way, so it is rated low severity. Severity counts across strategies: High 3 rows (all TEST029 claim_type in B1, B2, B3), Medium 22, Low 89. The most consequential real defect is the not_a_claim extraction rule, which comes from the prompt wording and is cheap to fix.

7. Recommendation

Use B3 (few-shot, k=8). It has the highest claim_type accuracy (98.9%), the best strategy-level record exact match (44.4% strict, 81.7% location normalised), the best extraction F1 (0.914) and the fewest failures (14 rows, 13 distinct records). Injury and high-urgency recall are 100%. It costs about $0.19 per 1,000 claims, 1.8 times zero-shot, and uses about 2,057 prompt tokens per call. At that price the extra accuracy is worth paying for in a triage setting.

When to choose otherwise. B2 (k=5) has the best urgency accuracy (98.3%) at $0.157 per 1,000 claims, but loses policy-number recall on not_a_claim messages. A is the cheapest ($0.104) and still never misses an injury, but over-escalates urgency and misses third-party liability, so it needs human review of every high-urgency item. C is not recommended: it costs about as much as B3, is slowest, and is less accurate.

Before deployment. (1) Reword the not_a_claim instruction and rerun B3 on the 60 test records. (2) Add examples for passenger injury, missing vehicles and multi-incident messages. (3) Route messages the model labels theft or not_a_claim with a missing vehicle to a person. (4) Normalise the gold location convention before comparing location scores.

8. Limitations

1. 60 test records is small; one record moves a score by 1.7 points per run, so close results are not conclusive. 2. The messages are synthetic and short. Real claims are longer, noisier and include attachments. 3. One annotator wrote the gold labels. The independent review of 20 labels is prepared (worksheet and comparison script in the repository) but has not been completed, so inter-annotator agreement is not reported. 4. A single model (gpt-oss-20b), a single temperature and low reasoning effort were tested; results may differ for other models or settings. 5. Latency was measured on a shared endpoint and is noisy. Cost is projected from list prices on 2026-11-13 and may change. 6. Few-shot examples were added in a fixed order, so the effect of each example cannot be separated from the effect of count. 7. Prompt D (optional) was not implemented. 8. Several gold labels (TEST029, TEST047, TEST051, urgency for destroyed vehicles) are judgement calls under the guide; different rules would shift the scores.

Reproduce: *python analysis.py --input-cost-per-million 0.075 --output-cost-per-million 0.30*, then *python scripts/build_error_analysis.py* and *python scripts/build_report.py*. No step calls the API.